

Hosting an LLM-Judge Suite

Amazon Bedrock AgentCore versus Arize AX, evaluated against a ten-judge helpfulness suite with structured verdicts, three-level scoring and k=5 aggregation.

TECHNICAL COMPARISON · V1.0 · BOTH INTEGRATIONS BUILT AND TESTED OFFLINE; NEITHER HAS RUN AGAINST A LIVE TENANT

Summary. Both platforms can host the suite, and both collapse a rich verdict into three fields. AgentCore wins decisively on one axis that turns out to decide the rest: because a code-based evaluator is *your own Lambda*, the existing Anthropic pipeline runs unchanged — schema-enforced output, reasoning effort, prompt caching, and k=5 with a median all survive. Arize's equivalent path was disqualified because it has nowhere to put an API key.¹ That single asymmetry retires the fatal limitation of the Arize integration and inverts a verdict reached three weeks earlier on the same suite.

THE ONE-LINE DIFFERENCE

Arize hosts the *prompt*; AgentCore hosts the *process*. Hosting a prompt means accepting the host's model configuration, output shape and repetition policy. Hosting a process means accepting only its input and output contract — which is a far smaller surrender.

1. What both platforms take away

THE SHARED COMPROMISE

Neither platform will store a structured verdict. Arize's eval record is exactly `name, score, label, explanation`²; an AgentCore code-based evaluator returns exactly `label, value, explanation`³. In both cases the judges' six-to-seven binary checks, evidence spans, confidence and per-judge extras must be serialised into the one free-text field.

Both integrations therefore use the same encoding — a compact JSON payload inside `explanation` — and both are, as far as published examples go, doing something neither vendor documents.

2. Feature comparison

MEASURED AGAINST THE SUITE'S ACTUAL REQUIREMENTS

REQUIREMENT	ARIZE AX	AGENTCORE	CONSEQUENCE
Structured verdict	3 fields	3 fields	Tie — both need payload-in-explanation
Schema enforcement	None; model-authored string, validated only after the fact ²	Full <code>json_schema</code> at generation time, because the Lambda calls Anthropic	AgentCore. Arize cannot detect a judge emitting five checks instead of seven until you parse it
Reasoning effort	Unreachable. <code>InvocationParamsRequest</code> has Gemini and OpenAI controls, none for Anthropic ⁴	Native — you build the request	AgentCore. This was graded <i>fatal</i> for Arize; the suite runs at <code>effort: high</code>
k=5 + median	No repetition primitive; duplicates rejected, re-runs replace ⁵	Inside the Lambda, with spread and review-routing preserved	AgentCore. Arize loses <code>unanimous</code> , <code>hard_failure_votes</code> and boundary-split routing
Prompt caching	No <code>cache_control</code> surface; it is a message-block annotation and the host builds the messages	Preserved. k=5 repeats an identical prefix five times	AgentCore. Ten judges share one 132-line preamble
Three-level scale	<code>classification_choices</code> takes an arbitrary label→score map ⁶	<code>ratingScale.numerical</code> , up to 20 definitions ⁷	Tie — both express 0 / 0.5 / 1.0 natively
Turn judge sees whole conversation	Via a pre-rendered span attribute you write	Native at TRACE level ⁸ , but the Lambda re-renders anyway	Tie in practice — both need instrumentation
Tool-call reach	Span scope cannot read sibling TOOL spans; subquery mapping is UI-only	Raw <code>sessionSpans</code> delivered to the Lambda	AgentCore. Two-directional trace reconciliation is only possible here
Declarative config	None. No Terraform provider, no <code>apply -f</code>	CloudFormation, Terraform and CDK resource types ⁹	AgentCore. Arize needs a reconcile script you own
Query the check vector	Filters are equality/comparison only; export required	Logs Insights has <code>parse</code> and <code>regex</code>	AgentCore, pending empirical confirmation
Secrets for a custom judge	No secret store; key would be plaintext and permanently versioned ¹	Lambda execution role + Secrets Manager	AgentCore. This is the root cause of most rows above
Execution ceiling	Host-managed; no user-visible timeout	Lambda 300 s max; 6 MB input cap ¹⁰	Arize. k=5 concurrent Opus calls must fit — the top sizing risk
Batch API discount	Not on the hosted path	Not on the online path	Tie — both keep it for offline anchor runs

REQUIREMENT	ARIZE AX	AGENTCORE	CONSEQUENCE
Setup burden	Instrument one attribute per span	Account-wide Transaction Search, conforming OTel spans, two IAM roles	Arize. AgentCore's prerequisite is plausibly the larger half of the work
Evaluator cap	No documented per-task cap	10 per online config, non-adjustable	Arize. The suite fits with zero headroom
Rubric edits	New version, task keeps running	Evaluator locked while its config is ENABLED	Arize. AgentCore needs pause → update → resume

THE CORRECTION WORTH RECORDING

The Arize integration graded Anthropic reasoning-effort as **fatal** — the suite runs every judge at **effort: high**, and Arize's schema exposes reasoning controls for Gemini and OpenAI but none for Anthropic.⁴ The AgentCore design retires that limitation entirely, not by finding a setting, but by moving the model call out of the host. Any comparison that scores only "does the platform have an eval feature" misses this.

3. Where each platform is genuinely better

HONEST PROS AND CONS

AgentCore is better at

Fidelity. Everything the suite depends on survives: schema, effort, caching, k=5, tool reach.

Automation. Real IaC resource types rather than a bespoke reconcile script.

Queryability. Logs Insights can parse the payload; Arize filters cannot.

Secrets. An execution role instead of a plaintext key.

Arize is better at

Time to first score. No account-wide prerequisite, no Lambda, no IAM.

Operational simplicity. No 300-second ceiling, no 6 MB input cap, no evaluator lock.

Headroom. No 10-evaluator cap to design around.

Purpose-built UI. Eval-first product rather than an observability console.

THE TRADE STATED PLAINLY

AgentCore buys fidelity with operational complexity. If the chatbot already runs on AgentCore and emits conforming spans, that complexity is mostly already paid and AgentCore is the clear choice. If it does not, Arize reaches a first score materially sooner — at the cost of the reasoning effort the suite was validated with, which means the numbers are not comparable to the anchor set.

4. Recommendation

AND THE CONDITION THAT WOULD CHANGE IT

Run the suite on AgentCore if the chatbot is on AgentCore or its traces already reach CloudWatch as conforming OTel spans. The code-based Lambda path preserves the judges as validated; no other hosted option does.

Keep Arize if it is already the observability system of record, and accept that its judges are a lower-fidelity sentinel — a shallow pass/warn/fail on sampled traffic, not a substitute for the validated suite.

Running both is defensible and cheaper than it looks, since the rubrics are byte-identical across the two integrations and only the tail differs. What is not defensible is averaging their scores: they are different instruments.

What would change this recommendation

Three empirical results, none yet obtained. **One** — if `k=5` at `effort: high` does not fit inside 300 seconds, AgentCore's advantage narrows to `k=3` and the gap closes. **Two** — if a ~1.5 KB JSON payload does not survive AgentCore's `explanation` field intact, both platforms lose the check vector and the comparison becomes about cost and operations alone. **Three** — if Arize adds an Anthropic reasoning parameter, its fatal limitation becomes minor and its simplicity advantage starts to dominate.

5. Method and limitations

WHAT THIS COMPARISON IS AND IS NOT

API claims were verified by reading shipped client libraries rather than documentation: the Arize 8.50.0 wheel (model fields, enums, regexes, constants) and botocore 1.43.83 service models for `bedrock-agentcore-control`. In both cases the published docs disagreed with the shipped code in at least one material place.

Neither integration has been run against a live tenant. Prompt assembly, schema derivation, span reconstruction, payload validation and request validation are tested offline — 49 checks for Arize, 51 for AgentCore. No judge has scored a production trace on either platform. Cost figures are not included because neither has been measured; the token bill will dominate either platform's own fee at the suite's volume, and that comparison should be made with real traffic.

References

1. Arize `CustomCodeConfigRequest` exposes `[data_granularity, query_filter, type, name, code, imports, variables, static_params]` — no `llm_config`, no secret store. Verified against the arize 8.50.0 wheel.
2. Arize `Evaluation.__properties = ["name", "score", "label", "explanation"]`; `EVAL_EXPLANATION_MAX_STR_LENGTH = 10_000`. Verified against `arize/_generated/api_client/models/evaluation.py` and `arize/constants/spans.py`, 8.50.0.
3. AgentCore code-based evaluators return `label` (required), `value`, `explanation`. AWS Bedrock AgentCore Developer Guide, Evaluations.
4. Arize `InvocationParamsRequest: thinking_level` (Gemini 3.x), `thinking_budget` (Gemini 2.5), `reasoning_effort` (OpenAI o-series/GPT-5). No Anthropic equivalent. Verified against the 8.50.0 wheel.
5. Arize `TaskEvaluatorInput.evaluator_id: "Duplicates are not allowed"`; `override_evaluations` replaces rather than appends. Verified against the 8.50.0 wheel.
6. Arize `TemplateConfigInput.classification_choices: Dict[str, float]`. Verified against the 8.50.0 wheel.
7. AgentCore `RatingScale` union — `numerical` (label, value, definition) or `categorical`; max 20 definitions. AWS Bedrock AgentCore Developer Guide.

8. AgentCore `CreateEvaluator.level` enum `[TOOL_CALL, TRACE, SESSION]` ; trace-level `{context}` carries all prior turns. Verified against botocore 1.43.83 service model.
9. `AWS::BedrockAgentCore::Evaluator` and `AWS::BedrockAgentCore::OnlineEvaluationConfig` CloudFormation types, with `hashicorp/aws` Terraform and CDK equivalents.
10. AgentCore `lambdaConfig.lambdaTimeoutInSeconds` bounds `min=1, max=300` . Verified against botocore 1.43.83 service model.
11. Zheng, L. et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." NeurIPS 2023 D&B. arXiv:2306.05685. — Full conversation in one prompt; judges mis-locate prior turns otherwise.
12. Lee, Y. et al. "CheckEval: A reliable LLM-as-a-Judge framework using checklists." EMNLP 2025. arXiv:2403.18771. — +0.45 cross-evaluator agreement from binary decomposition; the reason the check vector matters enough to preserve.
13. "The Coin Flip Judge? Reliability and Bias in LLM-as-a-Judge Evaluation." arXiv:2606.13685. — 13.6% verdict flip rate on repeated identical input; the reason k=5 is not optional.
14. Kim, S. et al. "Prometheus: Inducing Fine-grained Evaluation Capability in Language Models." ICLR 2024. arXiv:2310.08491. — Rubric contract; Pearson 0.897 with human evaluators.

Generated from arize-integration/ and agentcore-integration/. Both suites: proposed, not validated.